

Session 1: Environment Setup and First Automated Flow

Chapter 1: What is n8n and why is it changing the rules of the game in QA & DevOps?	2
1. The Evolution of Automation: From Scripts to Nodes	2
2. Basic elements of the n8n ecosystem	2
3. Why n8n and not commercial alternatives?	3
Did you know that...?	3
Applied Story: "Assembly Line"	4
🧠 Thinking Exercise: "The Architect's Eye"	4
Chapter 2: Installing the Engine - Node.js and n8n Locally (Zero Cost)	5
1. Verifying and Installing Node.js	6
2. Installing the n8n ecosystem	6
3. Security Bypass (SSL) - Environment Setting	7
4. Starting the Engine!	8
✅ Solutions and Answers	10
Chapter 3: Data Source and First Trigger - Connecting to Google Drive	12
1. How do systems communicate? Polling vs. Webhook	12
2. Practical Activity: Connecting to Google Drive	13
✅ Solutions and Answers	25

Chapter 1: What is n8n and why is it changing the rules of the game in QA & DevOps?



In this chapter, we debunk the myth that advanced automation necessarily requires thousands of lines of code. We explore the architecture of the n8n platform, understand the difference between a "hardcoded" code and a visual flow (Node-Based), and find out why a self-hosted solution is the "Holy Grail" for data security in enterprises.

1. The Evolution of Automation: From Scripts to Nodes

For QA and DevOps engineers, automation has traditionally meant writing complex scripts in Bash, Python, or Java. While extremely powerful, this approach comes with major limitations in large teams:

- **Cumbersome maintenance:** A script written by a colleague who left the company quickly becomes a "black box" that no one wants to touch.
- **Development time (Time-to-Market):** Integrating 3 different systems (e.g. Jira, Slack, and an AWS server) requires reading dozens of pages of API documentation and manually managing complex authentications.
- **Lack of visibility:** When a deployment or test script crashes, you have to dig through miles of log files to find the error.

This is where it comes in. **n8n** (pronounced *n-eight-n*). It is a workflow automation platform that transforms programming logic into visual components.

2. Basic elements of the n8n ecosystem

Instead of writing code sequentially, line by line, in n8n we "draw" the logic of the architecture.

- **The Node:** It represents a single action or connector to an external API. Think of nodes as smart Lego pieces. There's a node for Google Drive, one for Discord, one for running Terminal commands on the OS, etc.
- **The Wire/Connection:** The visual thread that connects two nodes on the canvas. This connection dictates the order of execution, but also *data flow* (Data Flow) from left to right.
- **JSON (JavaScript Object Notation):** It is the "native language" of the platform. All data flowing through connections, from one node to another, is formatted as JSON objects (a structured list of keys and values), making it extremely easy for humans to read and for the system to manipulate.

3. Why n8n and not commercial alternatives?

As technical engineers, you may ask: why wouldn't we use Zapier or Make.com, which are already famous?

- **Security and Control (Self-Hosted):** Zapier is strictly a SaaS (Software as a Service). If you automate sensitive data logs, customer databases, or source code, your data goes through their public servers. n8n, on the other hand, can be installed *locally* (on your laptop) or on your company's secure server. The data never leaves the internal network!
- **The Fair Code model:** You can run the platform for free at full capacity for internal or learning use.
- **Absolute flexibility:** Unlike rigid tools, if there is no predefined node for your application in n8n, you can directly write native JavaScript code in the "Code" node or call any API in the world using the universal "HTTP Request" node. You are only limited by your imagination.

Did you know that...?

💡 Although n8n is a visually friendly ("Low-Code") platform, behind it is built entirely on Node.js and TypeScript? This means that it benefits from all the speed and asynchronous architecture of the V8 engine (the same engine that runs Google Chrome), allowing it to process massive volumes of data and thousands of events per second without crashing!

Applied Story: "Assembly Line"



Imagine n8n as an assembly line in a state-of-the-art car factory.



Trigger (Start Node): It is the optical sensor at the factory entrance. It stays closed and only activates when a new car (an event/file) enters the lane.



Action nodes: These are the robotic arms. The first arm scans the car (reads the data), the second checks if it is painted correctly (an IF/ELSE logic node), and the third sticks the price tag on it (a node that sends a message on Discord).



JSON: It is the electronic "Roadmap" attached to the respective machine. Each robotic arm reads the map to know what it needs to do and adds new information to it before sending it to the next arm.



Thinking Exercise: "The Architect's Eye"

Consider the following tedious manual process that a QA colleague does daily:

"Every morning at 9:00 AM sharp, the tester opens Jira, searches for all 'Blocker' priority bugs created in the last 24 hours, copies their titles, and posts them on the Developer team's Slack channel to be resolved urgently."

Your task:How would you translate this process into Lego pieces (Nodes) in the n8n interface?

- What would Node 1 (the Trigger) be and how would you briefly configure it?
- What would be Node 2 (Data Source/Query System)?
- What would be Node 3 (Final Action)?

(The correct answers are 1. Schedule Trigger / Cron job at 09:00. 2. Jira Software Node - Get Issues. 3. Slack Node - Send Message. We will discuss/verify them at the end of the session).

Chapter 2: Installing the Engine - Node.js and n8n Locally (Zero Cost)



In this chapter, we move on to action. We leave the theory aside and open the command line. We will prepare the infrastructure on your laptop, technically explaining each command so that you understand exactly what you are running, not just Copy-Paste.

To run n8n on your own laptop (at zero cost) and have full control over your data, we need the engine it was written on: **Node.js**.

1. Verifying and Installing Node.js

Open the terminal (Command Prompt / PowerShell on Windows, or Terminal on Mac/Linux) and type:

```
node -v
```

- **If you receive a version** (e.g. v20.11.0 or newer), you're ready!
- **If you get an error**, means you need to download and install Node.js from the official website (nodejs.org). *Attention for Windows:* make sure that during installation you leave the option checked "Add to PATH".

2. Installing the n8n ecosystem

Once we have Node.js, we will use its package manager to download n8n. Run the following command in the terminal:

```
npm install -g n8n
```

Let's dissect this command, line by line:

- **npm (Node Package Manager):** It is the tool in Node.js that deals with finding, downloading, and installing software from the internet (it's like an App Store for programmers).
- **Install:** The clear command we give to npm to install a package.
- **-g (Global):** This "flag" tells the operating system to install n8n *onto* computer. Without it,

n8n would only install itself in the current folder. Thanks to -g, you will be able to start n8n from anywhere.

- **n8n:** The exact name of the package we want to download from public servers.

```

30 {
Problems Output Debug Console Terminal Ports

v22.17.1
• PS C:\Users\George\dev\code-lms> cd ..
• PS C:\Users\George\dev> npm install -g n8n

added 1 package in 887ms
• PS C:\Users\George\dev> npm install -g n8n
npm warn ERESOLVE overriding peer dependency
npm warn ERESOLVE overriding peer dependency
npm warn While resolving: @ai-sdk/anthropic@2.0.61
npm warn Found: zod@3.25.67
npm warn node_modules/n8n/node_modules/zod
npm warn peer zod@"^3.25.76 || ^4.1.8" from @ai-sdk/provider-utils@3.0.20
npm warn node_modules/n8n/node_modules/@ai-sdk/anthropic/node_modules/@ai-sdk/provider-utils
npm warn @ai-sdk/provider-utils@3.0.20 from @ai-sdk/anthropic@2.0.61
npm warn node_modules/n8n/node_modules/@ai-sdk/anthropic
npm warn
npm warn Could not resolve dependency:
npm warn peer zod@"^3.25.76 || ^4.1.8" from @ai-sdk/anthropic@2.0.61
npm warn node_modules/n8n/node_modules/@ai-sdk/anthropic
npm warn @ai-sdk/anthropic@"2.0.61" from n8n@2.22.6
npm warn node_modules/n8n
npm warn
npm warn Conflicting peer dependency: zod@4.4.3
npm warn node_modules/zod
npm warn peer zod@"^3.25.76 || ^4.1.8" from @ai-sdk/anthropic@2.0.61
npm warn node_modules/n8n/node_modules/@ai-sdk/anthropic
npm warn @ai-sdk/anthropic@"2.0.61" from n8n@2.22.6
npm warn node_modules/n8n

```

3. Security Bypass (SSL) - Environment Setting

Since we will be running the platform locally (your address will be <http://localhost>), the browser and n8n may block certain advanced functionalities (such as connecting your account to Google Drive via OAuth2) because we are not using a secure https protocol.

As engineers, we know that we are safe on our local environment, so we need to tell the platform to temporarily ignore this rule. In the same terminal, run the command specific to your operating system:

- **On Mac/Linux:**

```
export N8N_SECURE_COOKIE=false
```

- Pe Windows (CMD):

```
set N8N_SECURE_COOKIE=false
```

- Pe Windows (PowerShell):

```
$env:N8N_SECURE_COOKIE="false"
```

What does this command do?

Create an Environmental *Variable* (Environment Variable). This is an invisible "tag" attached to your terminal session. When n8n starts up right after, it will read the operating system, see this tag, and know to disable the strict security restrictions on web cookies.

4. Starting the Engine!

Now everything is ready. Simply run in the terminal:

```
n8n
```

The terminal will display the n8n ASCII logo and tell you that the server is running. Leave the terminal open (it's the "brain" now) and open your favorite browser. Go to:

<http://localhost:5678>

```
Failed to refresh the Registry
Version: 2.22.6
Building workflow dependency index...
Finished building workflow dependency index. Processed 0 draft workflows, 0 published workflows.

Editor is now accessible via:
http://localhost:5678

Press "o" to open in Browser.
```

localhost:5678/setup



Set up owner account

Email *

First Name *

Last Name *

Password *

8+ characters, at least 1 number and 1 capital letter

☐ I want to receive security and product updates

Next

Congratulations! You are in the automation visual interface. The first thing it will ask you to do is set up a local admin account.

💡 Did you know that...?

💡 n8n uses a database internally **SQLite** to save all your automations, credentials and execution history. That's why it's so easy to run it on your laptop: you don't have to install, configure and maintain cumbersome databases like MySQL or PostgreSQL. Everything sits compactly in a simple file hidden on your hard drive!

🧱 Applied Story: "The Foundation and the Smart Home"

🧠 Imagine you want to build a fully automated home.

- **Node.js**: It is the cement foundation. Without this solid foundation poured directly into the ground (the operating system), the house cannot be built.
- **n8n**: It is the actual smart home that you "build" (install) on top of the foundation.
- **npm (Package Manager)**: It's the building materials store (like Dedeman/Brico Depot) that delivers the "n8n package" from the manufacturer directly to your construction site (laptop) for free.

🕵️ Thought Exercise: "The Terminal Detective"

Analyze the following everyday situation in a testing team:

A QA colleague is trying to install n8n. He opens the Command Prompt on Windows and types `npm install -g n8n`. Immediately, the terminal returns this error in red text:

'npm' is not recognized as an internal or external command, operable program or batch file.

Your task: Playing detective (Troubleshooting), what step did your colleague miss before running the command and how do you help them solve the problem?

✅ Solutions and Answers

Solution to the "Terminal Detective" exercise:

The colleague missed **Step 1** The operating system clearly tells it that it doesn't know the word npm. This means that:

1. Either he doesn't have Node.js installed on his computer at all.
2. Either (the most common scenario on Windows) installed Node.js, but at the installation steps **forgot to check the "Add to PATH" box** Without this setting, Windows doesn't know where the npm executable is located on your hard drive.

Solution: They need to download the Node.js installer again, run the installation process, and make sure the "Add to PATH" (or "Add to environment variables") option is enabled. Then, they need to restart the terminal.

Chapter 3: Data Source and First Trigger - Connecting to Google Drive



In this chapter, we bring our automation to life. A workflow needs a starting point (an event). We will learn how to make n8n communicate securely with the Google ecosystem and understand the architectural difference between "asking" and "listening" in the world of APIs.

To extract technical documentation or test plans, we will use **Google Drive**. However, Google doesn't let just anyone read your files. We need a digital "ID" for n8n, called OAuth2.

1. How do systems communicate? Polling vs. Webhook

Before adding the first node, as engineers we must choose *MECHANISM* by which n8n learns that a new file has appeared in the cloud. There are two fundamental methods in the systems architecture:

Characteristic	Polling (Repeated Question)	Webhook ("Push" Notification)
Who initiates the discussion?	The client (n8n)	External Server (ex: Google / Slack)
How does it work?	n8n asks from minute to minute: <i>"Do you have a new file? What now?"</i>	n8n is not sitting idle. Google sends him a signal: <i>"Hey, a file has appeared!"</i>
Reaction time	Depends on the interval (e.g. reacts after max. 1	Instant (Real-Time)

	minute)	
Resource consumption	High (generates unnecessary traffic and queries)	Very low (only activated at event)

For Google Drive, n8n natively uses the mechanism **Polling**, checking the folder at regular intervals that we will define (e.g. every minute).

2. Practical Activity: Connecting to Google Drive

For this activity, we need to create a secure bridge (OAuth2) between our local machine (where n8n is running) and Google servers. Due to Google's strict security measures, we will go through the steps in a specific order.

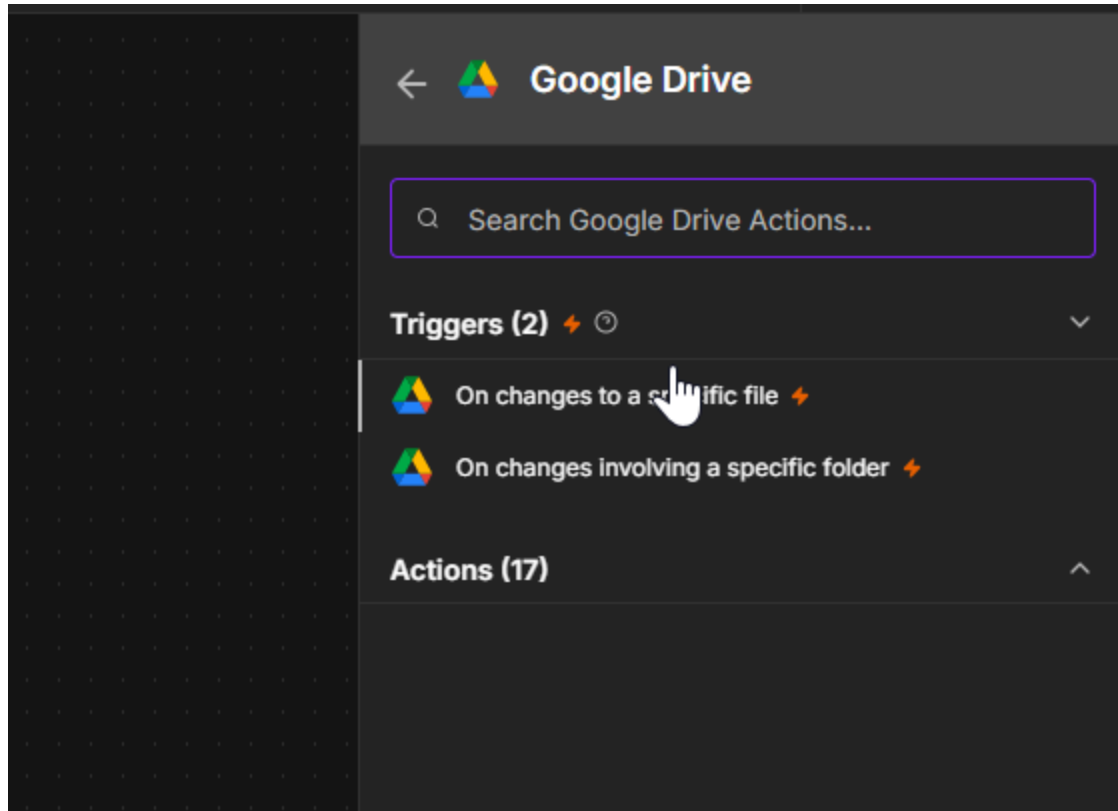
Step 0: First interaction with n8n

- Open your browser and access <http://localhost:5678>.
- Create your own **local account** by administrator (this data remains only on your computer).
- The "Canvas" will open in front of you - your visual workbench.

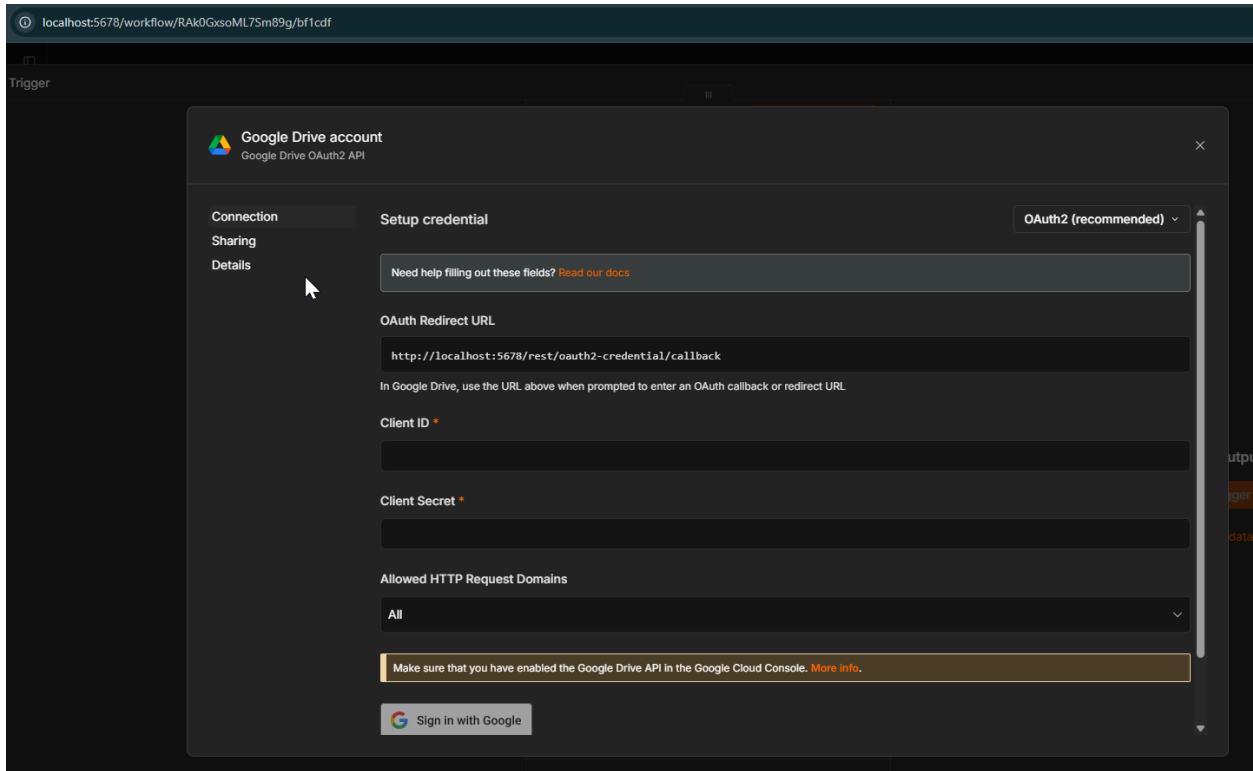
Step 1: Preparing the ground in n8n

Before we go to Google, we need to find out the address to which **Google** no way **send** the answer.

- On the canvas, click the button **Add first step** (Add the first step).
- Search and select **Google Drive**.
- As an action (Trigger), choose: **On specific changes to a file or folder** (On specific changes to a file/folder).



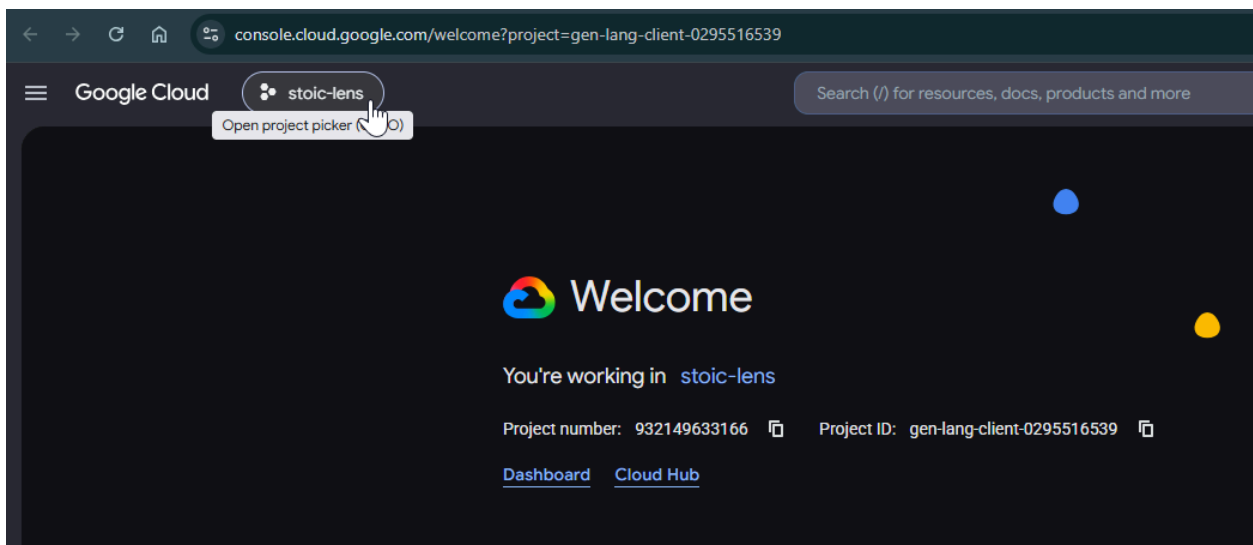
- In the credentials section, select **Create New Credential** and choose the type **OAuth2 (recommended)**.
- In the window that opens, copy the link from the section **OAuth Redirect URL** (usually it is: <http://localhost:5678/rest/oauth2-credential/callback>). Keep this link in your clipboard.

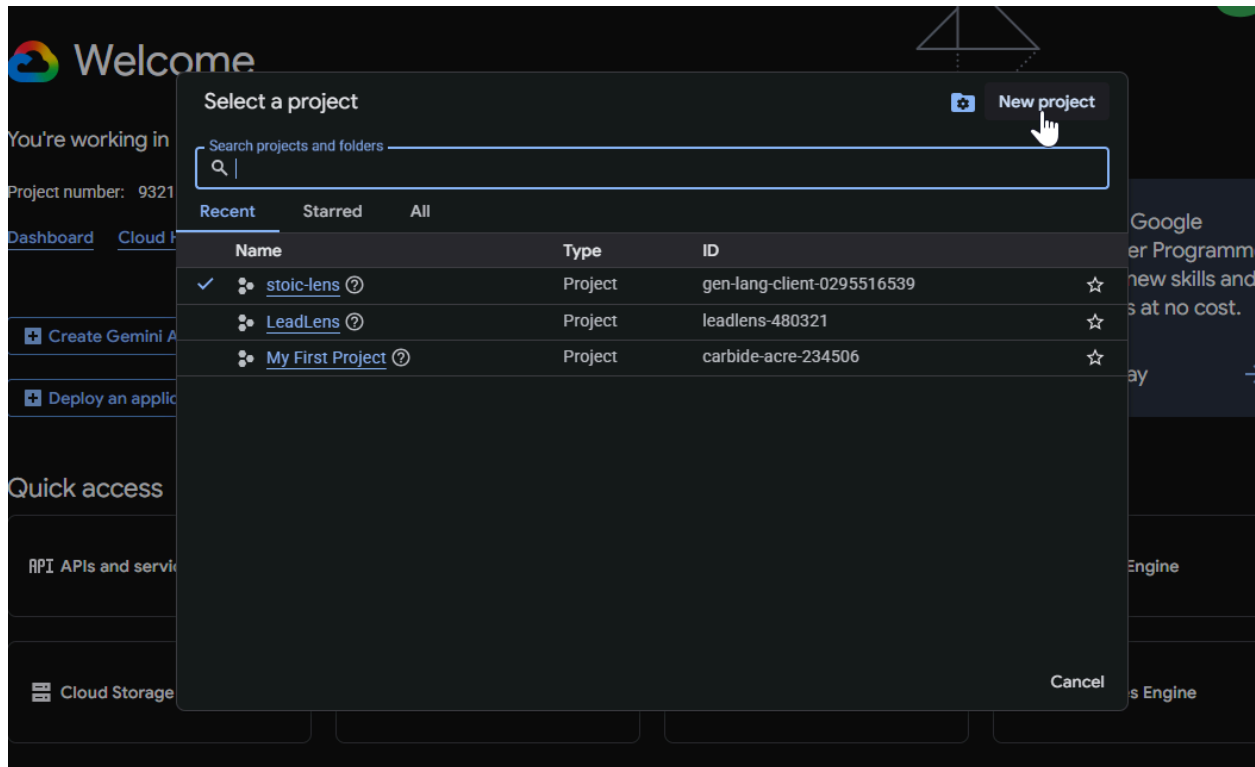


Step 2: Configuring the Google Cloud Console

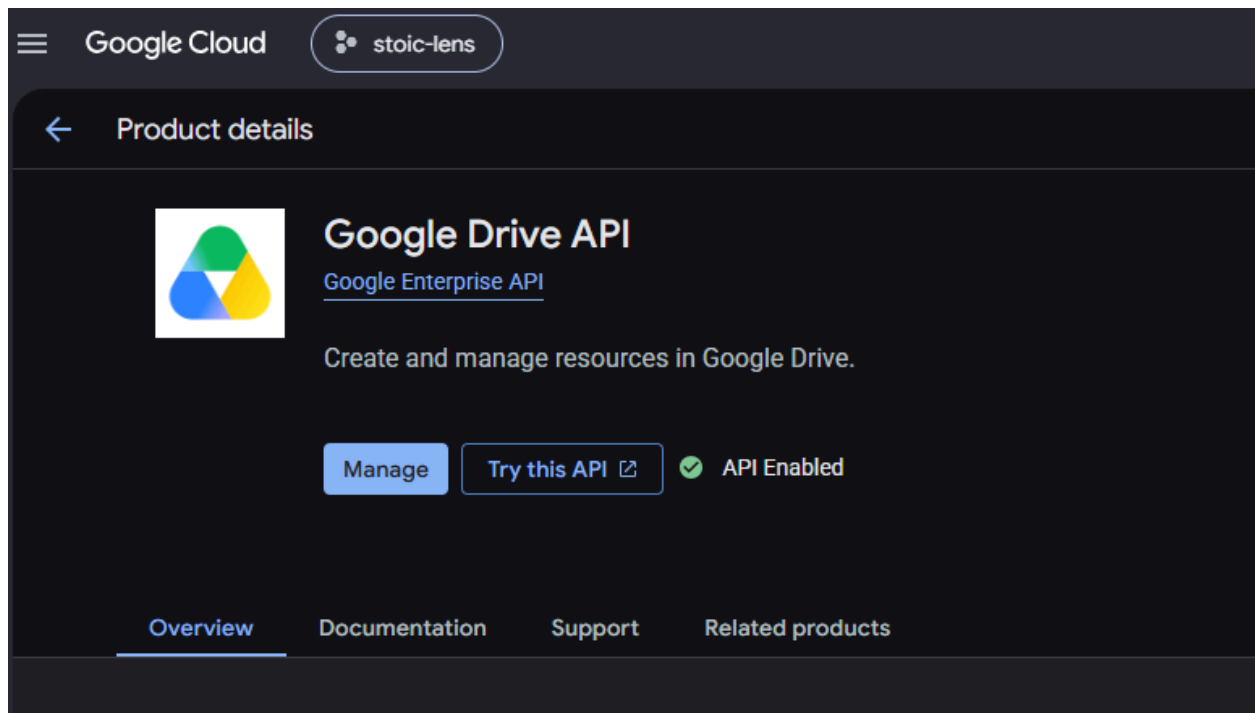
Now that we know the return address of n8n, let's go tell Google to trust it.

- Access [Google Cloud Console](#) and log in with your Google account.
- Click on **Select a project** (in the top bar) -> **New Project**, name it "n8n-QA-Automation" and hit Create.





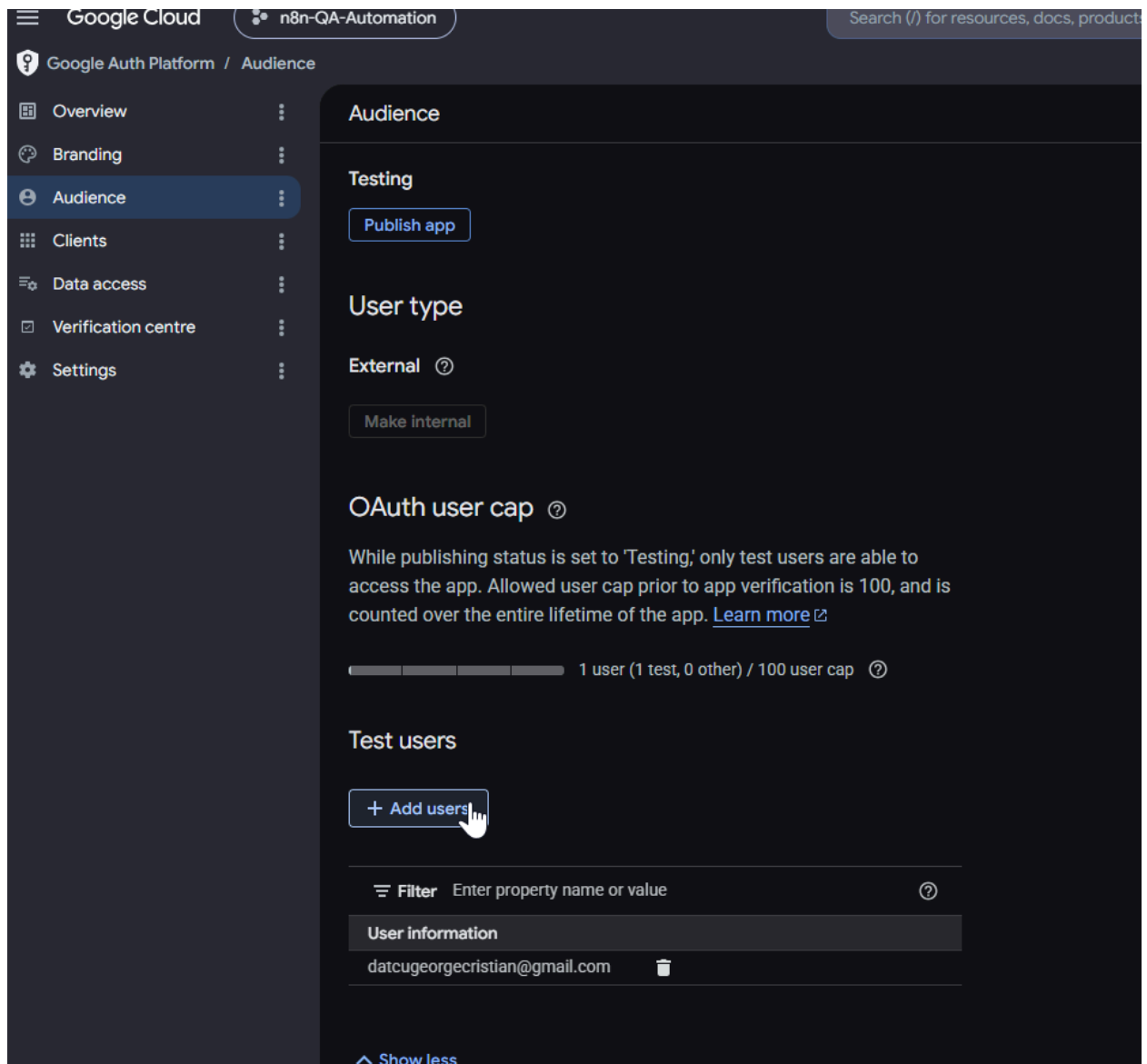
- Search in the top bar **Google Drive API** and press the button **Enable** (Activate).



Step 3: Avoiding the "Access Denied" Error (Vital!)

Google recently updated the security interface. If we skip this step, we won't be able to connect!

- From the main menu (left), go to **APIs & Services** -> **OAuth consent screen** (or *Google Auth Platform* if you have the new interface).
- Go to section **Audience** (Audience) located in the left menu.
- Because your application is in the development stage „**Testing**“, Google will automatically block access. Scroll until you find the section **Test users**.
- Click on **+ ADD USERS** and enter your exact email address that you use to log in to Google. Click **Save**.



Step 4: Generating Secret Keys

Now we create the actual connection.

- From the left menu, go to **Clients**(or Credentials) -> **Create Credentials** -> **OAuth client ID**.
- At Application type choose **Web application**. Call it "n8n local".



Create OAuth client ID

runs on multiple platforms, each will need its own client ID. See [Setting up OAuth 2.0](#) for more information. [Learn more](#) about OAuth client types.

Application type *

Web application

Name *

n8n Drive Auth

The name of your OAuth 2.0 client. This name is only used to identify the client in the console and will not be shown to end users.



The domains of the URIs you add below will be automatically added to your [OAuth consent screen](#) as [authorised domains](#).

Authorised JavaScript origins



For use with requests from a browser

+ Add URI

Authorised redirect URIs



For use with requests from a web server

URIs 1 *

http://localhost:5678/rest/oauth2-credential/callback



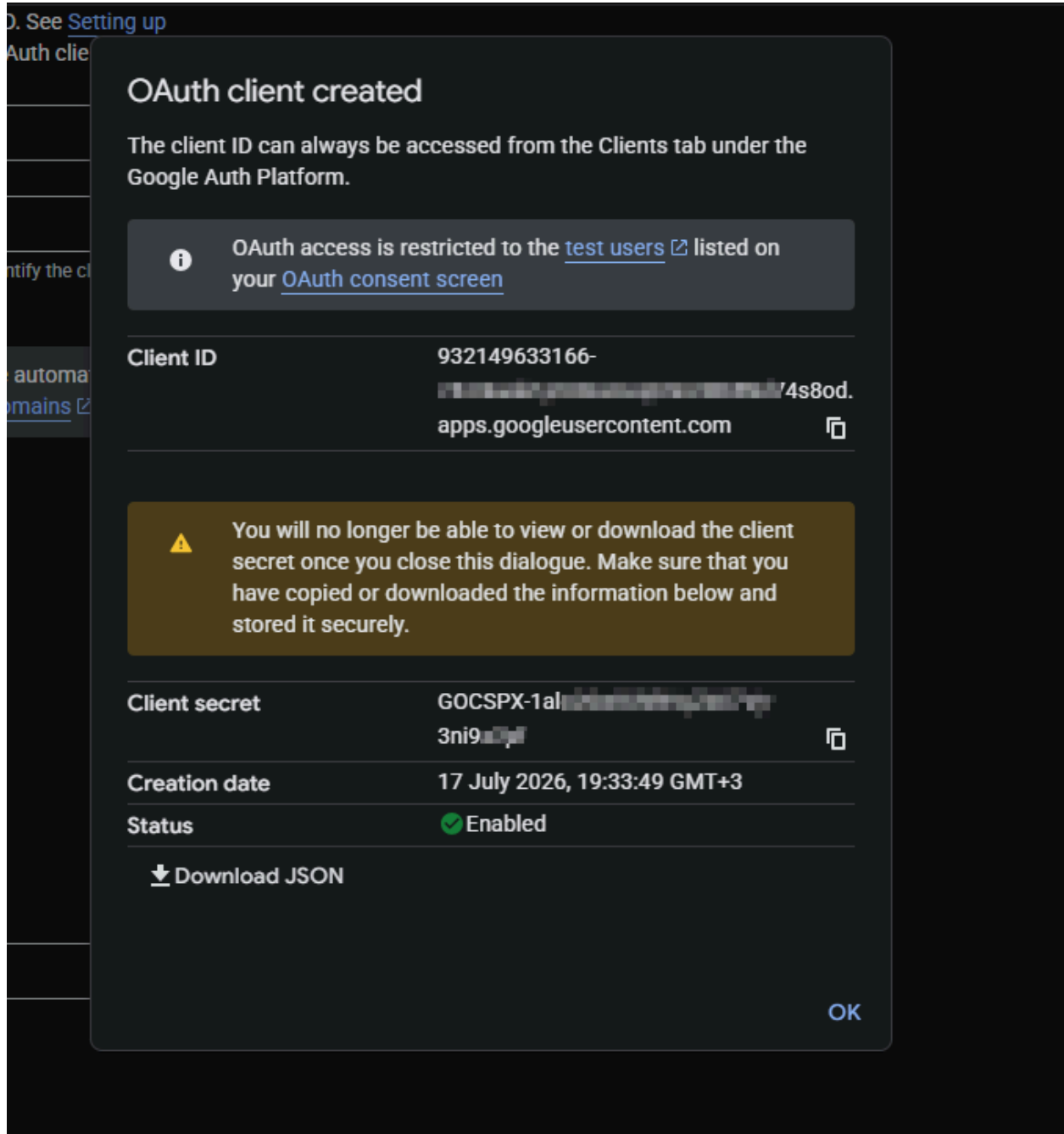
+ Add URI

Note: It may take five minutes to a few hours for settings to take effect

Create

Cancel

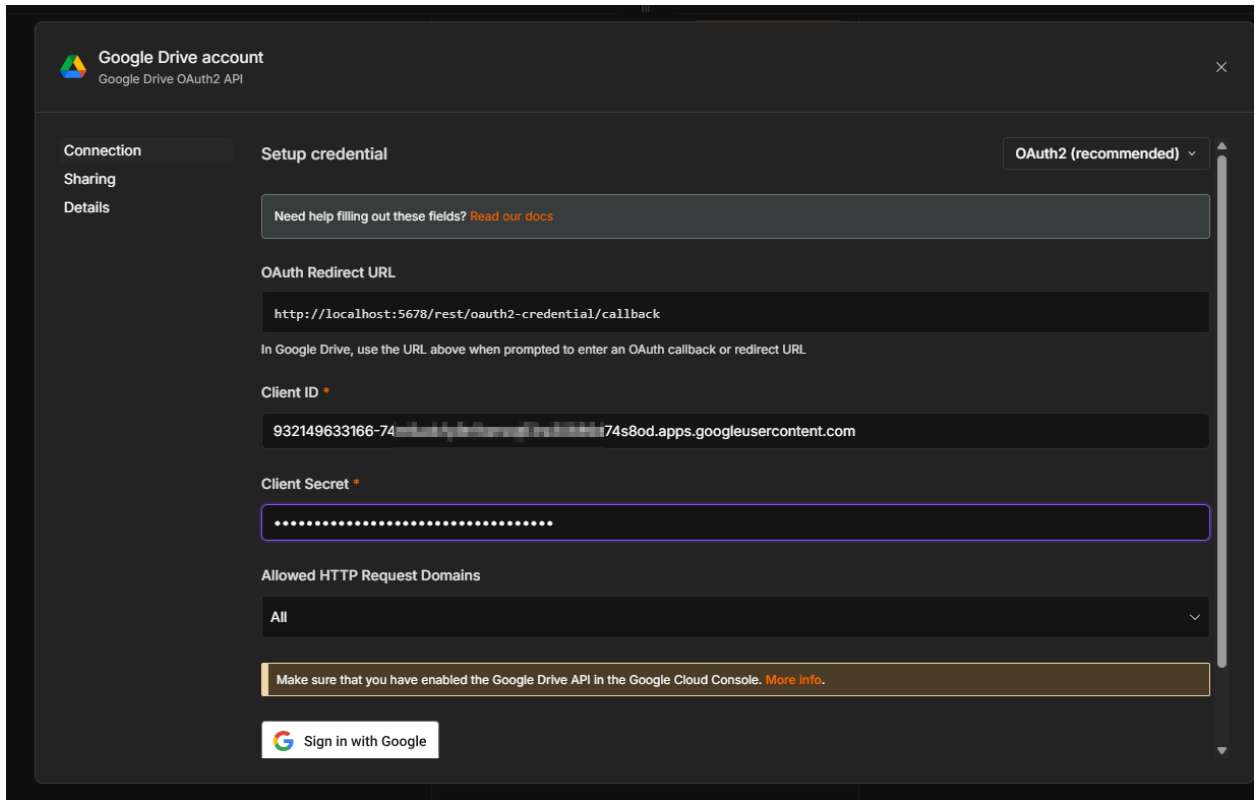
- To the section **Authorized redirect URIs**, press *Add URI* and **paste the link copied from Step 1 of n8n**.
- Press **Create**. Google will give you a **Client ID** and a **Client Secret**.



Step 5: Final Connection

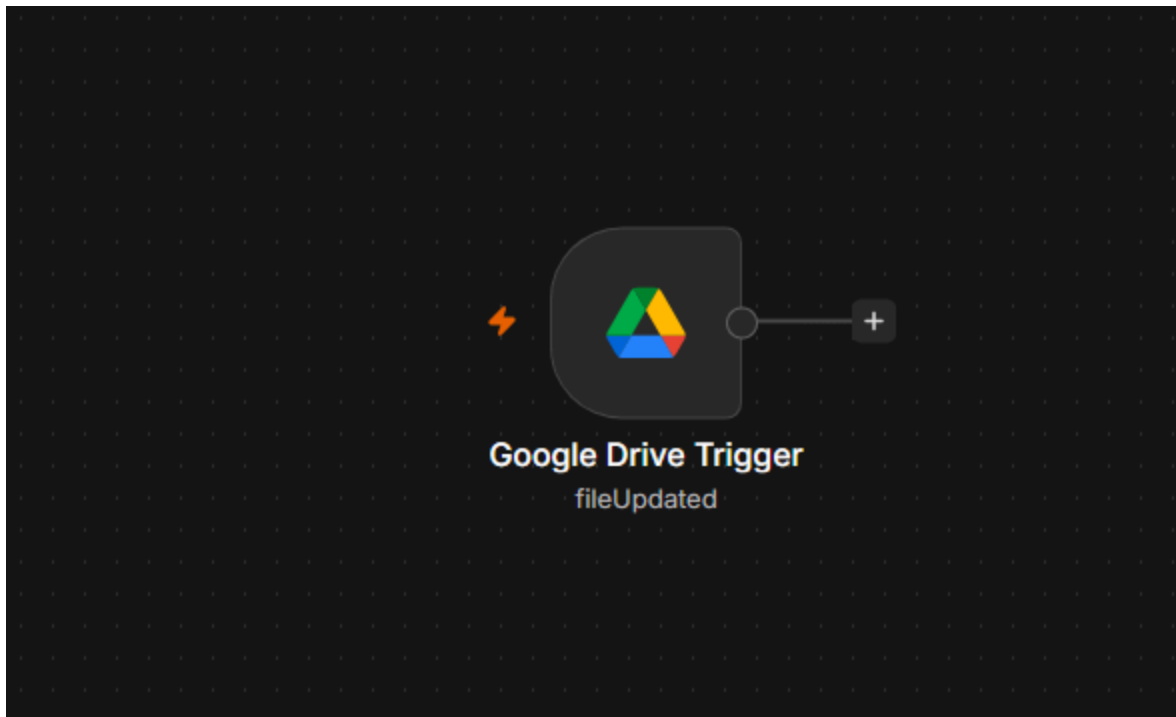
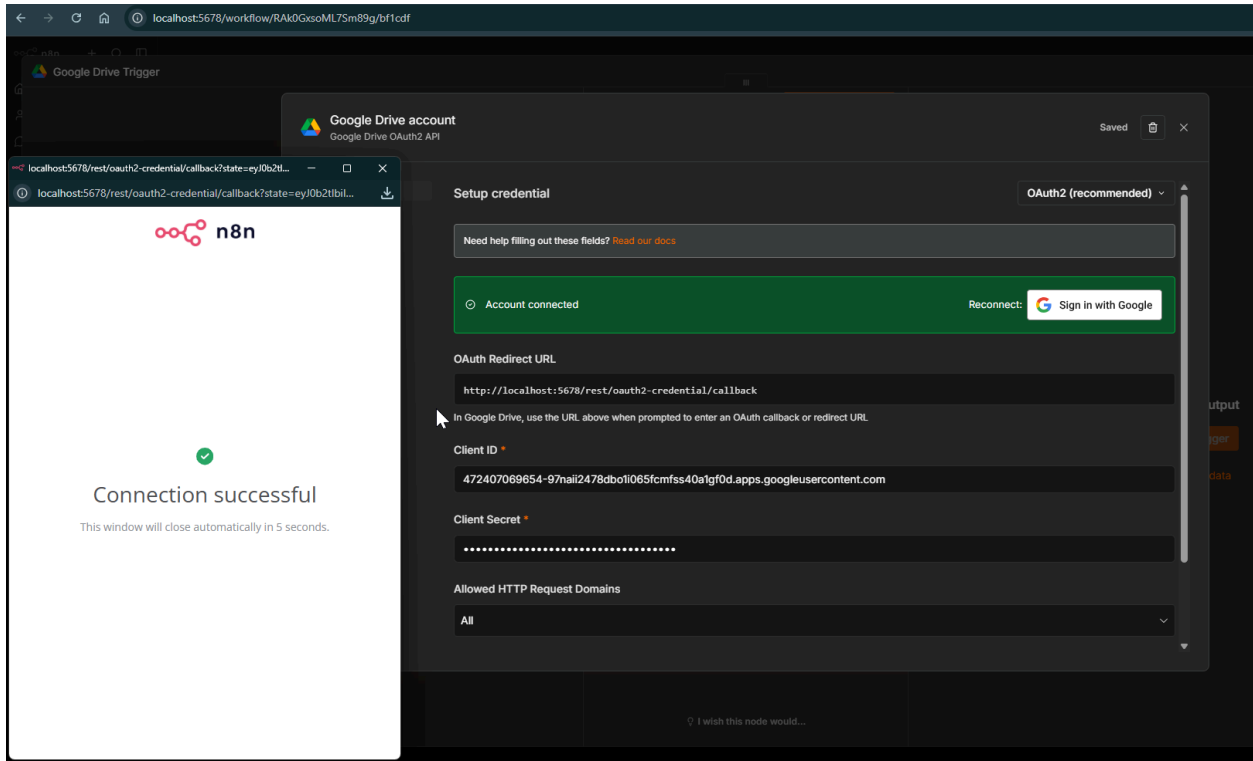
- Return to the n8n window (where you left off in Step 1).

- Enter **Client ID-ul** and **Client Secret-ul** in the appropriate boxes.
- Press the green button **Sign in with Google**.



The screenshot shows the 'Google Drive account' setup window for the 'Google Drive OAuth2 API'. The window has a dark theme and a sidebar on the left with tabs for 'Connection', 'Sharing', and 'Details'. The 'Connection' tab is active, showing the 'Setup credential' section. A dropdown menu at the top right is set to 'OAuth2 (recommended)'. Below this, there is a link to 'Need help filling out these fields? Read our docs'. The 'OAuth Redirect URL' is set to 'http://localhost:5678/rest/oauth2-credential/callback'. A note states: 'In Google Drive, use the URL above when prompted to enter an OAuth callback or redirect URL'. The 'Client ID' is '932149633166-74...74s8od.apps.googleusercontent.com'. The 'Client Secret' is masked with dots. The 'Allowed HTTP Request Domains' is set to 'All'. A yellow warning bar at the bottom says: 'Make sure that you have enabled the Google Drive API in the Google Cloud Console. More info.' At the bottom center is a 'Sign in with Google' button.

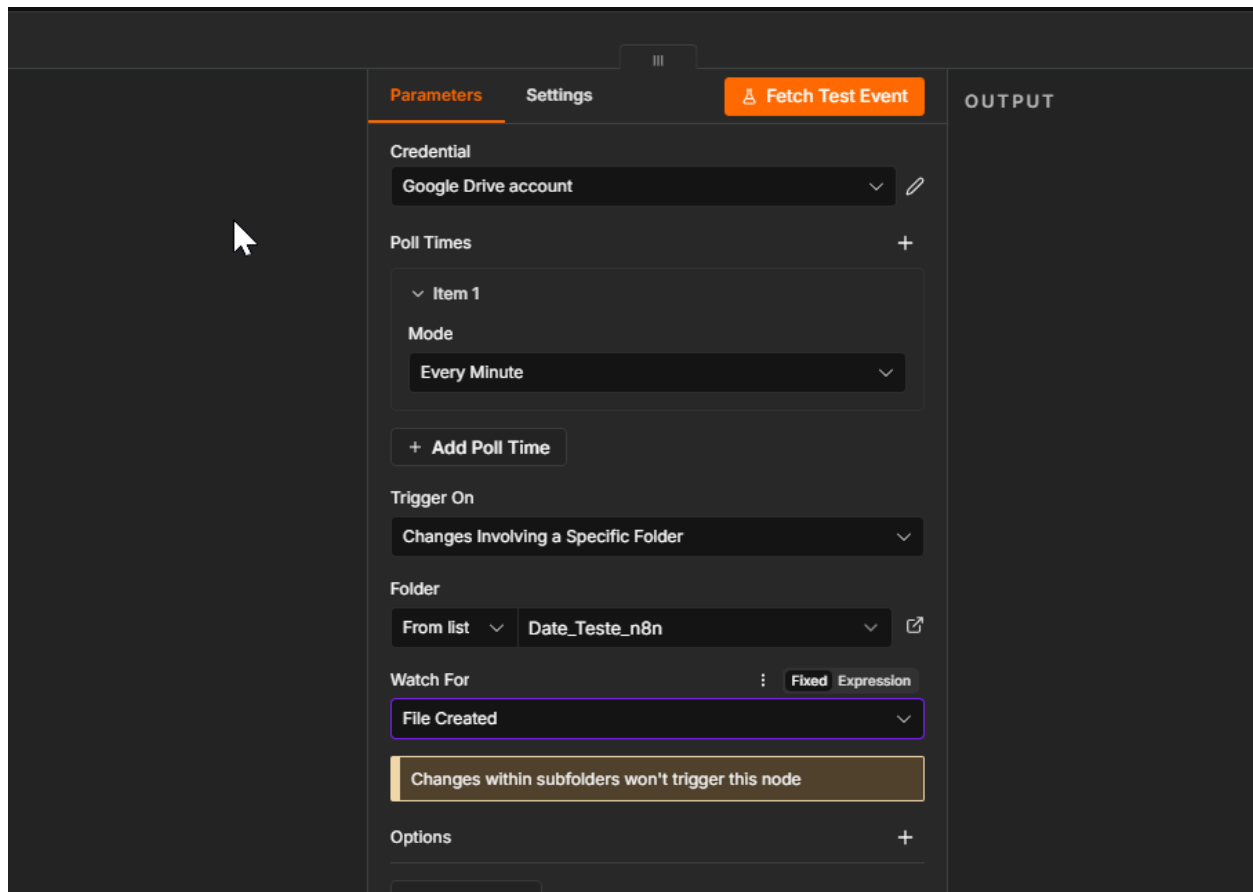
- Authorize access from the Google window that will appear (approving permissions to access Google Drive). Ignore the "Google hasn't verified this app" warning by clicking *Advanced* and then *Go to n8n-QA-Automation (unsafe)*, because it is your locally created application.
- The message „**Connection successful**” will appear on the screen! You've successfully completed your first complex API integration without writing a single line of code.



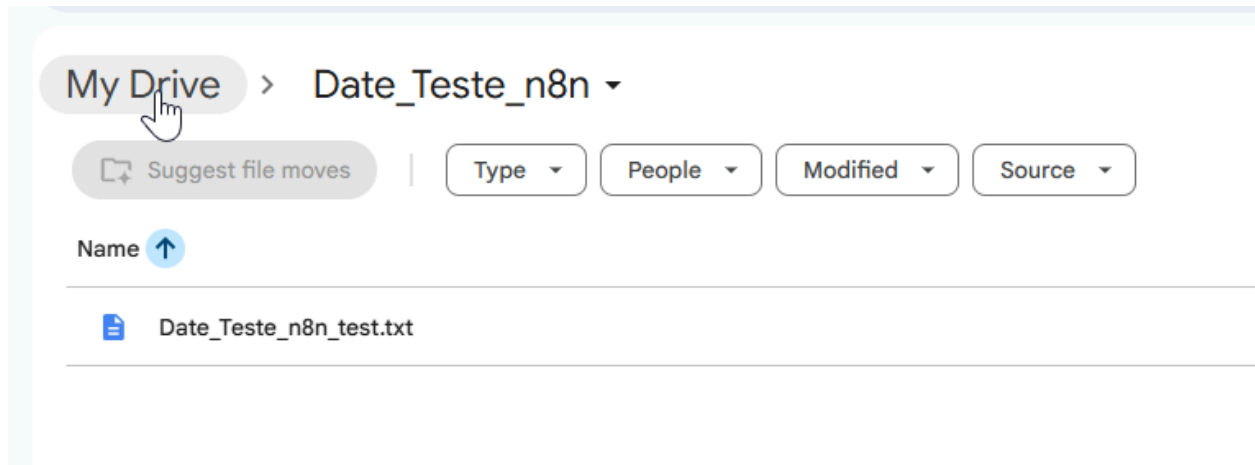
Step 6: First query (Fetch Test Event)

Now that we have the secure connection, we need to test whether the Trigger detects real files.

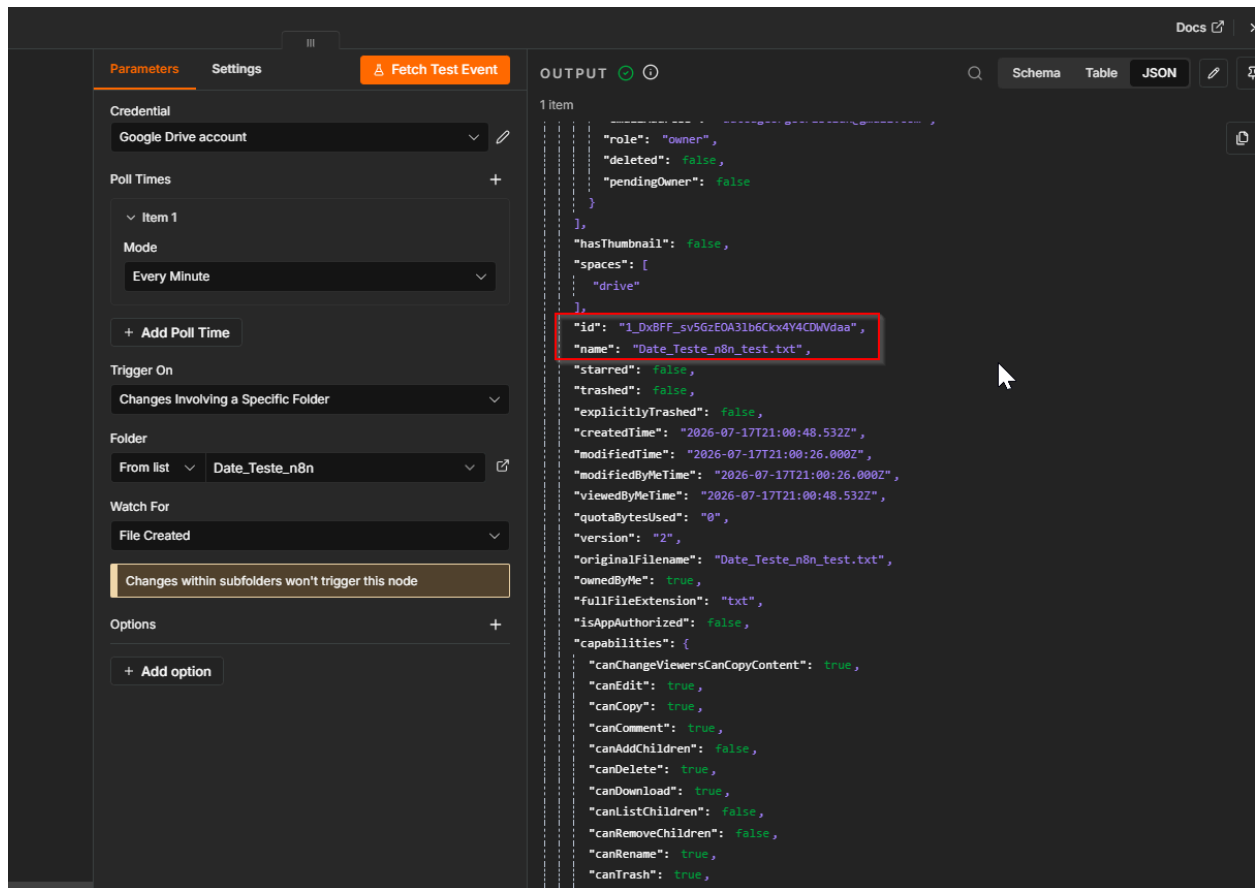
- Close the credentials window successfully and return to the Google Drive node settings in n8n.
- At the field *Trigger On* (Trigger on), choose In a Specific Folder.
- At the field *Folder*, use the selection button to choose a folder from your Drive (or create a new one, e.g. "Date_Teste_n8n"). In the field *Watch For*, make sure File Created is selected.



- Manual action: Go to another browser tab directly to your Google Drive and upload a test file (e.g. a PDF or TXT file) to that folder. Wait 5-10 seconds for Google to index the file.



- Go back to n8n and press the Fetch Test Event button (or *Test Step*).
- Magic! In the right pane (Output) you will see your original file data in JSON format (file name, unique ID, creation date). Your trigger is now "alive" and listening!



💡 Did you know that...?

💡 Most Enterprise integrations fail not because of the code, but because of poor credential management. n8n doesn't save your Client Secret in plain text on disk, but encrypts it using a unique key automatically generated upon first installation. That's why it's a favorite solution in security-oriented DevOps departments.

🧱 Applied Story: "The Postman and the Mailbox"

To make it easier to remember the difference from point 1:

- **Polling:** Are you (n8n) walking down the stairs of your apartment building every 5 minutes to check your mailbox (Google Drive) to see if your favorite magazine has arrived? You're wasting energy, and most of the time the mailbox is empty.
- **Webhook:** It's as if the Postman (the External System) had your phone number. You sit relaxed on the couch, without consuming any energy. When the magazine arrives in the box, the Postman sends you an SMS (HTTP POST Request). You come down at exactly the right time!

🧑 Thought Exercise: "Emergency Response"

Your infrastructure (DevOps) team uses an internal server monitoring application. You are asked to create an automation in n8n that restarts a test server. **IMMEDIATE** how the application detects a critical error (Crash).

Your task: As a QA/DevOps architect, which mechanism will you choose for the Trigger node of this automation: Polling every 1 minute or Webhook? Technically justify the decision.

✅ Solutions and Answers

Solution to the "Emergency Response" Exercise:

The only accepted technical answer is **Webhook**.

Justification: The requirement specifies the need for a reaction **immediately** (in real time). If we used Polling every 1 minute, there would be a risk that the server would crash at second 1, and the automation would only find out about it at second 59 (when it makes the next scheduled query). In a crisis situation on the infrastructure, 59 seconds of "downtime" can affect thousands of users. The webhook, being a "Push" notification system, will trigger the n8n flow exactly in the millisecond in which the monitoring application sends the error signal.